

Unsupervised Anomaly Detection Using Flow Matching on Tabular Data

Philip Konz¹, Tejaswini Medi¹, and Margret Keuper^{1,2}

University of Mannheim¹, MPI for Informatics, Saarland Informatics Campus²,
Germany
`philip.konz@gmx.de`

Abstract. Financial anomaly detection often relies on large unlabeled transaction logs, where anomalous samples may already be present during training. This training-set contamination violates the clean-normal data assumption used by many anomaly detection methods. Although flow matching has shown strong performance in generative modeling, its robustness for unsupervised tabular anomaly detection remains underexplored. In this work, we study flow-matching-based anomaly detection under contaminated training set by comparing Time-Conditioned Contraction Matching (TCCM) with Forest-Flow and evaluating multiple anomaly scoring functions. Our results show that the anomaly scoring function is critical: TCCM’s original single-step Decision score is sensitive to contamination, whereas trajectory-based Deviation and Reconstruction scores provide more stable anomaly signals, such that Forest-Flow becomes competitive with, and in some cases better than TCCM. Our findings highlight the importance of anomaly scoring for flow-matching in financial anomaly detection under severe anomaly class imbalance.

1 Introduction

Anomaly detection on tabular data is critical in high-stakes domains such as financial fraud detection [7], manufacturing fault detection [30], and medical diagnosis [5], where rare abnormal samples often correspond to costly or safety-critical events. Existing methods are broadly categorized into classical machine learning and deep learning approaches [13]. Classical methods, including Local Outlier Factor (LOF) [1], Principal Component Analysis (PCA) [27], and one-class Support Vector Machines (SVMs) [25], are widely used because of their simplicity but often struggle with high-dimensional and large-scale datasets due to limited scalability and reduced effectiveness in complex feature spaces [13]. Deep learning methods overcome some of these limitations by learning richer representations and have demonstrated strong empirical performance [21]. However, adversarial models such as AnoGAN [24] suffer from unstable min-max optimization, diffusion-based methods [18] require carefully designed noise schedules and slow iterative inference, and normalizing flows are constrained by invertibility requirements and Jacobian computations [19]. Many deep anomaly detectors also incur high inference latency, limiting their practicality on large-scale tabular data [13].

Another important limitation is that both classical and deep learning methods are typically evaluated in semi-supervised settings assuming clean normal training data [13]. In practice, however, training sets often contain unlabeled anomalies, and identifying or removing them is expensive [8]. Such contamination can substantially degrade detection performance [8]. Recent work has therefore explored robust strategies such as ensemble-based detectors, which reduce the influence of contaminated samples by aggregating diverse models [8], albeit at increased computational cost.

Flow Matching [15] has recently emerged as a powerful generative modeling framework that combines the stability and expressivity of diffusion models with more efficient deterministic sampling dynamics [16, 12, 13]. It learns a time-dependent vector field that continuously transports samples from a simple source distribution, such as Gaussian noise, to the target data distribution. Building on this framework, Time-Conditioned Contraction Matching (TCCM) adapts flow matching for tabular anomaly detection and achieves state-of-the-art performance in semi-supervised settings [13]. Instead of learning the full transport vector field, TCCM predicts a time-conditioned contraction vector that progressively moves samples toward the origin.

Despite its strong performance, TCCM is designed for clean normal training data, leaving its behavior under fully unsupervised settings with contaminated training sets largely unexplored. Moreover, although the original work reports weaker performance for conventional flow-matching models, this comparison is limited to the semi-supervised setting. Consequently, the robustness of flow-matching-based anomaly detectors under training-set contamination remains insufficiently understood.

In this work, we investigate flow-matching-based anomaly detection in the fully unsupervised setting. We first examine how different anomaly scoring functions affect the robustness of TCCM under contaminated training data and whether alternative scoring strategies mitigate performance degradation. We then evaluate whether conventional flow-matching models, specifically Forest-Flow [10], can match or outperform TCCM when paired with suitable scoring functions. Experiments are conducted primarily on two financial tabular datasets with markedly different anomaly ratios: Campaign ($\rho = 11.27\%$) and Synthetic Business Transaction ($\rho = 0.786\%$), enabling evaluation under both moderate and extreme class imbalance.

Our main contributions are as follows:

- We investigate flow-matching-based anomaly detection under contaminated training data, focusing on realistic unsupervised financial tabular datasets with unlabeled anomalies.
- We show that anomaly scoring is crucial for robustness: TCCM’s original single-step Decision score is highly sensitive to contamination, whereas trajectory-based scores, namely Deviation and Reconstruction, provide more stable anomaly signals.
- We demonstrate that trajectory-based scoring substantially improves robustness and detection performance. In particular, Deviation and Reconstruction

scores enable ForestFlow to become competitive with, and in some cases outperform, the state-of-the-art TCCM baseline across both moderate and severe anomaly class imbalance settings.

Our work is organized as follows. Section 2 introduces the proposed models and anomaly scoring functions. Section 3 describes the experimental setup. Section 4 presents the main results, Section 5 analyzes hyperparameter sensitivity, and Section 6 concludes the findings.

2 Methodology

We evaluate two flow-matching-based models with distinct design goals: ForestFlow [10], a general-purpose generative model for tabular data, and TCCM [13], a flow-based approach designed for tabular anomaly detection. Although both learn ordinary differential equation (ODE)-based velocity fields via flow matching, they differ in how they represent normality and derive anomaly scores. ForestFlow learns a generative transport from Gaussian noise to the data distribution, with anomaly detection performed post hoc by measuring how well a sample conforms to the learned generative dynamics. In contrast, TCCM learns a contraction field that pushes samples toward a fixed point, making deviations from the learned contraction behavior a direct anomaly signal.

For both models, we evaluate three anomaly scoring functions: Decision, Deviation, and Reconstruction. The Decision score corresponds to the original TCCM scoring function, which is Lipschitz-continuous with respect to the input [13]. It evaluates the velocity prediction error at a single representative time point: the contraction prediction error at the original input for TCCM, and the transport velocity prediction error near the data endpoint for ForestFlow. The Deviation score accumulates velocity prediction errors across multiple time steps along the learned flow trajectory, measuring the temporal consistency of the learned dynamics. The Reconstruction score numerically integrates the learned velocity field using Euler steps and measures the ℓ_2 reconstruction error at the expected endpoint, capturing errors that accumulate along the flow trajectory.

Notation. Let $x \in \mathbb{R}^d$ denote a tabular sample, where d is the feature dimension, and let $z \sim \mathcal{N}(0, I_d)$ denote standard Gaussian noise. The time variable is $t \in [0, 1]$, discretized as $t_i = i/(n_t - 1)$ for $i = 0, \dots, n_t - 1$. For Monte Carlo estimation in ForestFlow, we draw K independent noise samples $z^{(k)}$, $k = 1, \dots, K$, for each input sample.

2.1 Forest-Flow with XGBoost Regressor (FF-XGB)

ForestFlow [10] learns a velocity field under the conditional flow-matching framework [15]. During training, each data point x is paired with Gaussian noise z , and intermediate states are constructed by linear interpolation, $x_t = (1-t)z + tx$. The corresponding target velocity is the constant displacement $v_{\text{true}} = x - z$,

which defines the transport direction from noise to data. Forest-Flow discretizes time into n_t levels and trains a separate XGBoost regressor [3] $v_\theta^{(i)}$ at each time step t_i to predict this displacement from x_{t_i} . The training objective is

$$\mathbb{E} \left[\left\| v_\theta^{(i)}(x_{t_i}) - (x - z) \right\|_2^2 \right],$$

which encourages the learned velocity field to align with the conditional transport path. At test time, each sample x is paired with K independent noise samples $z^{(k)}$, yielding interpolated points $x_{t_i}^{(k)} = (1 - t_i)z^{(k)} + t_i x$. Since later time steps lie closer to the data distribution and are less dominated by noise, we evaluate anomaly scores on

$$\mathcal{T}_{\text{FF}} = \{t_i : i = \lfloor n_t/2 \rfloor, \dots, n_t - 2\}.$$

The Decision score measures the velocity prediction error at the last evaluated time step. The Deviation score accumulates velocity prediction errors across all $t_i \in \mathcal{T}_{\text{FF}}$. The Reconstruction score measures how accurately x can be recovered by numerically integrating the learned velocity field using Euler steps. Scores are averaged over the K noise realizations to obtain the final anomaly score for each sample. Formal definitions of the scoring functions are provided in Appendix section A.

2.2 Time-Conditioned Contraction Matching (TCCM)

TCCM [13] is not a generative noise-to-data model like Forest-Flow. Instead, it learns a deterministic contraction field that moves samples toward the origin. Given a sample x and time $t \in [0, 1]$, a single multi-layer perceptron (MLP) f_θ predicts a time-conditioned velocity using sinusoidal time embeddings. The training objective is

$$\mathbb{E}_{x,t} \left[\left\| f_\theta([x; \text{Embed}(t)]) + x \right\|_2^2 \right],$$

which encourages the predicted velocity to match the contraction direction $-x$. Thus, the origin acts as a stable fixed point of the learned dynamics.

Unlike Forest-Flow, TCCM uses one continuously time-conditioned network, does not learn a generative transport from noise to data, and does not require Monte Carlo sampling. For evaluation, we use the deterministic probe path $x(t_i) = (1 - t_i)x$ and evaluate early time levels

$$\mathcal{T}_{\text{TCCM}} = \{t_i : i = 0, \dots, \lfloor n_t/2 \rfloor - 1\},$$

where probe points remain close to the original sample. The Decision score measures contraction prediction error at the original input. The Deviation score accumulates prediction errors along the probe path. The Reconstruction score measures the error after numerically integrating the learned contraction field toward the origin. Formal definitions of the scoring functions are provided in Appendix section A.

3 Experiments

3.1 Datasets

We evaluate the models on three tabular anomaly detection datasets with different anomaly ratios and data characteristics.

Campaign is a public financial dataset from ADBench [6], with 41,188 samples, 62 numerical features, and 4,640 labeled anomalies, corresponding to an anomaly ratio of $\rho = 11.27\%$. This high-dimensional dataset serves as our primary financial benchmark and was also used in the original TCCM evaluation [13].

Synthetic Business Transaction is a synthetic financial transaction dataset modeling business-to-business (B2B) payment activity for nine customers. It contains 6,364 transactions with 50 labeled anomalies, corresponding to an anomaly ratio of $\rho = 50/6364 \approx 0.786\%$. Each transaction is encoded using 21 numerical, temporally derived features. The anomalies cover suspicious patterns such as amount spikes, duplicate payments, payee or IBAN changes, temporal shifts, and atypical payment channels. This makes the dataset suitable for evaluating anomaly detection under extreme anomaly class imbalance. Further details are provided in Appendix section C.

To assess generalization beyond financial data, we also evaluate **Waveform**, a physics dataset from ADBench [6] that was used in the original TCCM evaluation [13]. It contains 3,443 samples, 21 numerical features, and 100 anomalies, corresponding to anomaly ratio of $\rho = 2.90\%$.

3.2 Evaluation Protocol

We evaluate robustness under training-set contamination following [13]. For **Campaign**, normal samples are randomly split into equally sized training and test sets using seed 42. Anomalies are split evenly, with one half added to the fixed test set and the other half reserved for controlled injection into the training set. For **Synthetic Business Transaction**, we use a chronological midpoint split to preserve its time-series structure. Because anomalies are temporally non-uniform, this yields 28 anomalies in the training period and 22 in the test period. Features are z-score normalized using statistics estimated from the training set only and then applied to the test set to avoid information leakage [14].

We evaluate contamination levels of the training set from 0.0% up to each dataset’s anomaly ratio. Each model is trained with five random seeds (0–4), while the test set remains fixed. Performance is reported using Area Under the Receiver Operating Characteristic Curve (AUROC) and Area Under the Precision-Recall Curve (AUPRC) [13, 20]. AUROC measures anomaly detection ranking quality, with 1.0 being the perfect discrimination. AUPRC summarizes the precision–recall trade-off and is especially informative under severe anomaly class imbalance. Higher values indicate better performance for both metrics.

3.3 Hyperparameter Details

Forest-Flow. Following the large-dataset setting ($N > 10,000$) in [10], we use $n_t = 20$ time steps and $K = 20$ Monte Carlo noise samples per data point

The results in Table 1 show that anomaly score selection strongly affects robustness under training-set contamination. On Campaign, FF-XGB-Rec. achieves the best AUROC under both zero and full contamination (0.820 and 0.804), as well as the best zero-contamination AUPRC (0.341), while TCCM-Rec. gives the best full-contamination AUPRC (0.336). On the highly imbalanced B2B dataset, TCCM-Rec. obtains the best AUROC under both zero and full contamination (0.869 and 0.846). FF-XGB-Dev. achieves the best zero-contamination AUPRC (0.195), whereas FF-XGB-Rec. gives the best full-contamination AUPRC (0.094). On Waveform, FF-XGB-Rec. performs best across all metrics, achieving AUROC values of 0.741 and 0.706 and AUPRC values of 0.156 and 0.102 under zero and full contamination, respectively.

Overall, trajectory-based scores, particularly Reconstruction, provide more stable anomaly signals than the single-step Decision score. FF-XGB-Rec. performs best on Campaign and Waveform, while TCCM-Rec. achieves the highest AUROC on the B2B dataset. A possible explanation is that the Decision score depends on the prediction error at a single evaluation point, so any local bias introduced by contaminated training data directly affects the anomaly score. By contrast, the Deviation and Reconstruction scores accumulate prediction errors along the learned trajectory. If contamination primarily affects only parts of the learned vector field, aggregating information over multiple time steps reduces the influence of these local deviations and emphasizes the overall consistency of the learned dynamics. This makes trajectory-based scores more robust to training-set contamination. Additional contamination curves across all contamination levels are provided in Appendix section D.

5 Hyperparameter Sensitivity

Figure 1 summarizes the effect of the hyperparameters on AUROC and AUPRC across contamination levels. For Forest-Flow, increasing the number of noise samples (K) generally improves anomaly detection performance, with the largest gains for the Decision score. Reconstruction is less sensitive to K , suggesting that trajectory-based aggregation already provides stable anomaly estimates. Compared with the default setting ($K = 20$), using $K = 10$ (Figure 1) degrades the performance of the Decision and Deviation scores. In contrast, increasing the number of time steps (n_t) from 20 to 50 has only a minor effect, particularly for Deviation and Reconstruction.

For TCCM, reducing n_t from 20 to 5 improves the stability and mean performance of the Deviation score, whereas the Decision score is unaffected because it is always evaluated at $t = 1$, irrespective of n_t . For Deviation, a larger n_t extends the probe trajectory to increasingly shrunk points relative to the training inputs, introducing prediction noise that weakens the anomaly signal. A smaller n_t keeps the trajectory closer to the training distribution, improving robustness. Reconstruction is less affected because the finer Euler integration associated with a larger n_t partially offsets the impact of the more off-distribution probe points.

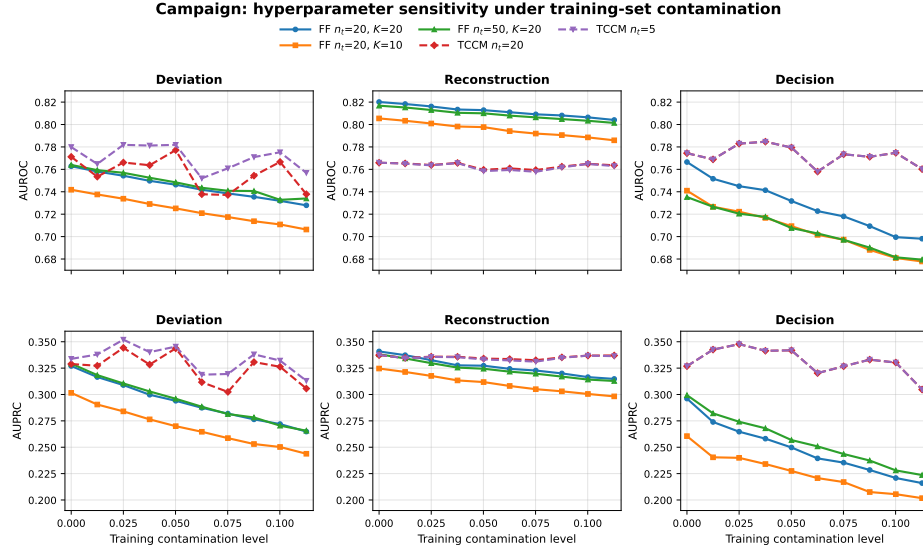


Fig. 1: Hyperparameter sensitivity on the Campaign dataset under increasing training-set contamination. AUROC and AUPRC are reported for the Deviation, Reconstruction, and Decision scores. For Forest-Flow, n_t denotes the number of time steps and K the number of Monte Carlo noise samples per data point. For TCCM, only n_t is varied because the model does not use Monte Carlo noise-sample averaging.

Overall, the Reconstruction score is the most stable across hyperparameter settings and contamination levels for both Forest-Flow and TCCM, consistently achieving strong AUROC and AUPRC. In contrast, the Decision score is the most sensitive to hyperparameter choices.

6 Conclusion

We studied flow-matching-based unsupervised anomaly detection under contaminated tabular training data, with a focus on financial applications where unlabeled anomalies may be present in transaction logs. Our results show that robustness depends not only on the underlying flow-matching model but also on the choice of anomaly score. While the original TCCM anomaly score is sensitive to training-set contamination, trajectory-based scores such as Deviation and Reconstruction provide more robust anomaly signals by aggregating velocity prediction errors along the learned flow trajectory. Moreover, Forest-Flow, originally proposed for tabular data generation, becomes a competitive anomaly detector when combined with trajectory-based scoring. These findings highlight the importance of anomaly score design when adapting flow-matching models for anomaly detection. Future work will investigate larger financial benchmarks and more computationally efficient trajectory-based scoring methods.

Appendix

A Scoring Function Definitions

This section describes the two flow matching frameworks formally. For each model, we describe the underlying framework and define three anomaly scoring functions for each model.

Notation. Let $x \in \mathbb{R}^d$ denote a data sample and $z \sim \mathcal{N}(0, I_d)$ a standard Gaussian noise vector, where d is the number of feature dimensions. The time parameter t indexes the transformation process. Discrete time levels are denoted t_i for $i = 0, 1, \dots, n_t - 1$, where n_t is the number of time levels. To obtain the expected values (Monte-Carlo estimate), we sample K (duplication factor) independent noise vectors $z^{(k)} \sim \mathcal{N}(0, I_d)$ for $k = 1, \dots, K$ to build interpolations with a test or training sample. θ denotes the set of learnable model parameters (e.g., XGBoost trees or neural network weights). The notation is consistent across different scores for one model.

A.1 Forest-Flow

Forest-Flow [10] is a tree-based implementation of conditional flow matching [15] originally proposed for tabular data generation. We adapt it for anomaly detection by training the model and evaluating three scoring functions. Note that the actual implementation of the Forest-Flow code [9] and the paper [10] deviate from their time setting and the defined noise levels. We take the framework of code implementation [9] as the scoring functions are adapted to it. Very minor details of the code implementation are disregarded for simplicity (i.e. numerical stability constant for time).

Framework Forest-Flow learns a time-dependent vector field that transports (scaled) samples from a standard Gaussian distribution to the data distribution [9]. It uses a linear interpolation path between noise $z \sim \mathcal{N}(0, I_d)$ (at $t = 0$) and data x (at $t = 1$):

$$x_t = (1 - t) \cdot z + t \cdot x, \quad t \in [0, 1] \quad (1)$$

The true velocity (training target) along this linear path is constant as proposed by [15]:

$$v_{\text{true}} = x - z \quad (2)$$

Forest-Flow trains a separate XGBoost regressor for each discrete time level t_i [10], uniformly spaced in $[0, 1]$:

$$t_i = \frac{i}{n_t - 1}, \quad i = 0, 1, \dots, n_t - 1 \quad (3)$$

This results in n_t models, where each model (here the XGBoosts; with θ as the parameters) $v_{\theta}^{(i)} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ takes an interpolated sample $x_{t_i} \in \mathbb{R}^d$ as

input and predicts the velocity vector. For notational convenience, we write $v_\theta(x_t, t_i) := v_\theta^{(i)}(x_t)$.

Each model $v_\theta^{(i)}$ is trained independently by minimizing the mean squared error between predicted and true velocity:

$$\mathcal{L}^{(i)} = \mathbb{E}_{x \sim p_{\text{data}}, z \sim \mathcal{N}(0, I_d)} \left[\|v_\theta^{(i)}(x_{t_i}) - (x - z)\|_2^2 \right] \quad (4)$$

where $x_{t_i} = (1 - t_i) \cdot z + t_i \cdot x$ is the interpolated sample at time t_i . The expectation is approximated by pairing each data point with K independent noise samples, creating $K \cdot n$ training pairs per time level, where n is the number of samples in the training set.

Scoring Functions for Forest-Flow All scores are computed in the preprocessed (scaled) space unless otherwise stated. Forest-Flow interpolates from noise ($t = 0$) to data ($t = 1$). We focus on later time levels for the Reconstruction and Deviation Score where the signal-to-noise ratio is higher (see formula of interpolation points):

$$\mathcal{T}_{\text{Flow}} = \{t_i : i \in \lfloor n_t/2 \rfloor, \dots, n_t - 2\} \quad (5)$$

The extreme index $i = n_t - 1$ (corresponding to $t = 1$) is excluded to avoid boundary effects where the interpolation reaches the data point exactly.

Decision Score. The Decision (inspired by the original anomaly scoring function of TCCM) evaluates prediction accuracy at a single time level very close to the data distribution at t_{n_t-2} (very high signal to noise ratio). For a test sample x and K independent noise samples $z^{(k)} \sim \mathcal{N}(0, I_d)$, the interpolated samples are:

$$x_{t_{n_t-2}}^{(k)} = (1 - t_{n_t-2}) \cdot z^{(k)} + t_{n_t-2} \cdot x \quad (6)$$

The Decision Score averages squared velocity prediction errors over all noise samples:

$$s_{\text{Decision}}^{\text{Flow}}(x) = \frac{1}{K} \sum_{k=1}^K \left\| (x - z^{(k)}) - v_\theta \left(x_{t_{n_t-2}}^{(k)}, t_{n_t-2} \right) \right\|_2^2 \quad (7)$$

Normal samples should yield small errors because the model has learned accurate velocity predictions for in-distribution samples. Anomalous samples produce larger errors as they lie outside the learned flow trajectories.

Deviation Score. The Deviation Score extends the Decision Score by accumulating prediction errors across multiple time levels in $\mathcal{T}_{\text{Flow}}$, providing a more comprehensive assessment of model fit along the trajectory:

$$s_{\text{Deviation}}^{\text{Flow}}(x) = \frac{1}{K} \sum_{k=1}^K \sum_{i=\lfloor n_t/2 \rfloor}^{n_t-2} \left\| (x - z^{(k)}) - v_\theta \left(x_{t_i}^{(k)}, t_i \right) \right\|_2^2 \quad (8)$$

where $x_{t_i}^{(k)} = (1 - t_i) \cdot z^{(k)} + t_i \cdot x$. The accumulation over multiple time levels amplifies systematic prediction errors for anomalous samples while averaging out random fluctuations for normal samples.

Reconstruction Score. The Reconstruction Score measures how accurately the model reconstructs the original data by following the learned vector field over the full trajectory from intermediate points to the final time $t = 1$. Starting from an interpolated point $x_{t_i}^{(k)}$, we solve the Ordinary Differential Equation (ODE) forward using Euler integration with step size $h = 1/(n_t - 1)$. We initialize

$$x_0^{(i,k)} = x_{t_i}^{(k)}, \quad \tau_0 = t_i \quad (9)$$

and iteratively update:

$$x_{j+1}^{(i,k)} = x_j^{(i,k)} + h \cdot v_\theta(x_j^{(i,k)}, \tau_j), \quad \tau_{j+1} = \tau_j + h. \quad (10)$$

Here, j indexes the Euler integration steps, i.e., $j = 0, \dots, (n_t - 1) - i - 1$, so that after $(n_t - 1) - i$ steps the time reaches $\tau_{(n_t-1)-i} = 1$. The Reconstruction Score is computed in the original data space after applying inverse preprocessing transformations:

$$s_{\text{Reconstruction}}^{\text{Flow}}(x) = \frac{1}{K} \sum_{k=1}^K \sum_{i=\lfloor n_t/2 \rfloor}^{n_t-2} \left\| \tilde{x} - \tilde{x}_{\text{end}}^{(i,k)} \right\|_2^2 \quad (11)$$

where $\tilde{\cdot}$ denotes the post-processed sample in original data space and $x_{\text{end}}^{(i,k)} = x_{(n_t-1)-i}^{(i,k)}$. Small prediction errors can compound during integration, making it sensitive to systematic model errors but also more computationally demanding. This score measures the environment of the learned vector field around the ideal trajectory of the test sample. Anomalous samples should have a more disturbed vector field around the ideal trajectory.

A.2 TCCM (Time-Conditioned Contraction Matching)

TCCM [13] is a neural network-based flow matching method [15] specifically designed for anomaly detection.

Framework TCCM [14, 13] learns a continuous time-conditioned contraction field that maps normal data toward the origin [13]. Importantly, although this behavior can be interpreted as a contraction dynamic, the method is not based on explicitly solving an ODE trajectory[13]. Instead, it uses a simplified supervision scheme with a fixed target direction $-z$ for each training sample z in all time steps.

Specifically, for $x \sim p_{\text{data}}$ and $t \sim \mathcal{U}(0, 1)$, the model takes the augmented input $\tilde{x} = [x; \text{Embed}(t)]$ and predicts a velocity vector $f_\theta(\tilde{x})$. f_θ is the MLP with θ as its learned parameters. The time is encoded with sinusoidal embeddings [28]. Training minimizes

$$\mathcal{L}_{\text{TCCM}} = \mathbb{E}_{x,t} [\|f_\theta([x; \text{Embed}(t)]) + x\|_2]. \quad (12)$$

Key Differences from Forest-Flow. TCCM differs from Forest-Flow in three main aspects: (1) TCCM learns a contraction velocity toward the origin while Forest-Flow learns a trajectory (ODE) between Gaussian and data distributions; TCCM is not a generative model (2) TCCM uses continuous time conditioning during training while Forest-Flow trains on discrete time levels; (3) TCCM uses one neural network (Multi-Layer Perceptron) as a Universal Function Approximator (UFA) for all time steps, while Forest-Flow uses XGBoost regressors [10] for each discrete time step. Due to deterministic training and inference, TCCM does not require Monte Carlo estimation ($K = 1$ implicitly).

Scoring Functions for TCCM While TCCM is trained with continuous time [13], our Deviation and Reconstruction Scores require discrete time levels for evaluation:

$$t_i = \frac{i}{n_t - 1}, \quad i = 0, 1, \dots, n_t - 1 \quad (13)$$

TCCM time levels start at $t_0 = 0$. For the Deviation and Reconstruction Scores, we construct a synthetic linear interpolation path toward the origin:

$$x(t_i) = (1 - t_i) \cdot x \quad (14)$$

This path is *not* a true ODE (Ordinary Differential Equation) trajectory (the model is not trained on that), but serves as a probe path for evaluation. For the Reconstruction and Deviation Score we use the first half of time levels where probe points remain closer to the original sample:

$$\mathcal{T}_{\text{TCCM}} = \{t_i : i \in \{0, \dots, \lfloor n_t/2 \rfloor - 1\}\} \quad (15)$$

We do this to be consistent with the Forest-Flow approach. Additionally, this seems to enhance detection performance (sparsely tested) but is, unlike Forest-Flow, not connected to a higher signal-to-noise ratio (there is no noise included at all).

Instead, enhanced performance might come from the Lipschitz continuity of f_θ (the MLP, see below) [13]: Near the original data points, the Lipschitz constraint guarantees a well-behaved vector field, making the Deviation Score more sensitive to actual anomalies rather than numerical instabilities that could arise further along the interpolation path. In general, the Lipschitz continuity is very critical by ensuring robustness guarantees under input perturbation [13]. Note that [13] proves that the scoring function in the paper (Decision Score for TCCM) is Lipschitz continuous, whereas this property is not guaranteed for Deviation and Reconstruction Score.

Another property of TCCM is its explainability [13], meaning that the anomalousness of a data sample can be traced back to a certain feature. This is possible because the residual vector itself encodes per-feature contributions to the anomaly score (Decision Score)[13]. Since residual vectors are calculated for Deviation and Decision Score as well (and then just averaged), these scoring function should also be interpretable.

Decision Score. The Decision Score is the original anomaly detection function from [13]. At $t = 1$, the ideal prediction for any point x is $f_\theta(x, 1) = -x$. Any other time between $(0, 1]$ could have been chosen [13]. The Decision Score measures the deviation from this ideal:

$$s_{\text{Decision}}^{\text{TCCM}}(x) = \|f_\theta([x; \text{Embed}(1)]) + x\|_2 \quad (16)$$

Samples that don't fit into the data distribution (anomalies) should yield a higher deviation [13]. Note that this uses the Euclidean norm, unlike all the other scores, which use squared errors.

Deviation Score. The Deviation Score probes temporal consistency of the learned contraction field at multiple interpolated positions along a synthetic path toward the origin:

$$s_{\text{Deviation}}^{\text{TCCM}}(x) = \sum_{i=0}^{\lfloor n_t/2 \rfloor - 1} \|f_\theta([x(t_i); \text{Embed}(t_i)]) + x(t_i)\|_2^2 \quad (17)$$

where $x(t_i) = (1 - t_i) \cdot x$. At each interpolated point, the true contraction vector should be $-x(t_i)$. For normal samples, interpolated points tend to stay within regions where the contraction rule $f_\theta(z, t) \approx -z$ is well-supported, yielding consistently small residuals. For anomalous samples, the same probe points are more likely to fall into poorly covered regions, producing larger accumulated residuals.

Reconstruction Score. The Reconstruction Score measures how accurately the learned flow transports interpolated points toward the origin when integrated forward. Starting from $x_0^{(i)} = x(t_i)$ at each time level $t_i \in \mathcal{T}_{\text{TCCM}}$ with $\tau_0 = t_i$, we integrate using Euler's method:

$$x_{j+1}^{(i)} = x_j^{(i)} + h \cdot f_\theta(x_j^{(i)}; \text{Embed}(\tau_j)), \quad \tau_{j+1} = \tau_j + h, \quad h = \frac{1}{n_t - 1}, \quad (18)$$

with $j = 0, \dots, (n_t - 1) - i - 1$. After $(n_t - 1) - i$ steps, the trajectory reaches $\tau = 1$, and we denote the terminal state by

$$x_{\text{end}}^{(i)} := x_{(n_t-1)-i}^{(i)}. \quad (19)$$

Because the intended terminal state at $\tau = 1$ is the origin, the Reconstruction Score accumulates endpoint deviations:

$$s_{\text{Reconstruction}}^{\text{TCCM}}(x) = \sum_{i=0}^{\lfloor n_t/2 \rfloor - 1} \|x_{\text{end}}^{(i)}\|_2^2. \quad (20)$$

For normal samples, the learned flow transports the interpolated points closer to the origin. For anomalous samples, small local prediction inaccuracies compound, leading to systematic endpoint deviations.

B Related Work

Unsupervised outlier detection (UOD) aims to identify anomalous samples without access to labeled data during training. Most deep UOD methods assume clean training sets [23, 24], an assumption that is frequently violated in large-scale scenarios where anomalies are inevitably present. This mismatch has motivated approaches that explicitly learn from contaminated datasets [31, 32]. As noted by [8], performance degradation under training-set contamination is a well-known issue. Accordingly, prior work distinguishes two settings for unlabeled anomaly detection. The semi-supervised setting assumes access to uncontaminated data containing only normal samples (e.g., DeepSVDD [23], NeuTraL AD [22], ICL [26], AnoGAN [24]). In contrast, the unsupervised setting operates directly on contaminated datasets, identifying anomalies within the training data or via adaptation to a test set (e.g., RandNet [2], ROBOD [4], RDA [31]). State-of-the-art deep unsupervised methods typically rely on ensemble strategies to improve robustness against contamination [2, 4, 17, 29]. These approaches are not considered in this work, as our focus lies on flow-matching models. The recently proposed approach TCCM [13], which uses a specialized flow-matching for semi-supervised anomaly detection, includes a contamination ablation, observing uniform performance degradation as contamination increases. However, it does not investigate whether alternative anomaly scoring strategies or conventional flow-matching formulations could mitigate this effect.

Flow-matching has emerged as a promising generative modeling framework, combining the training stability and expressivity of diffusion models with significantly lower computational cost [16, 12, 13]. It learns an ordinary differential equation (ODE) that deterministically maps samples from a source to a target distribution, enabling faster sampling and simpler optimization compared to diffusion models based on stochastic differential equations (SDEs) [13, 18]. Unlike diffusion models, flow-matching does not require a forward noising process, nor does it rely on explicit density estimation as in normalizing flows [11]. New samples are generated by evolving draws from the source distribution along the learned ODE toward the target distribution.

For tabular data, Forest-Flow [10] is a generative flow-matching model that predicts velocity fields at discrete time steps using XGBoost regressors and achieves strong performance on generation benchmarks. TCCM [13], a recent anomaly detection method, adapts flow matching for anomaly detection and differs from Forest-Flow in three key aspects: (i) TCCM learns a contraction velocity that maps data samples toward the origin rather than a generative trajectory from a Gaussian source to the data distribution; (ii) TCCM employs continuous-time conditioning during training, whereas Forest-Flow operates on discrete time levels; and (iii) TCCM uses a single shared multi-layer perceptron across all time steps, while Forest-Flow trains separate XGBoost regressors per time step. Due to its deterministic formulation, TCCM does not require Monte Carlo estimation ($K = 1$). While TCCM demonstrates strong empirical performance, alternative scoring functions beyond the original formulation have not yet been explored within its architecture.

C Synthetic Business Transaction Dataset

Table 2: Feature groups in the Synthetic Business Transaction dataset.

Group	Features
Amount	amount, amount_zscore_series, amount_ratio_to_mean
Timing	day_of_month, day_of_week, month, days_since_last_in_series, day_deviation_from_usual
Mismatch flags	iban_mismatch, swift_mismatch, method_mismatch, channel_mismatch, known_name_new_iban, is_card_mobile
Novelty	is_new_series, is_new_ref_name, is_new_iban
History	series_tx_count_before_log, ref_name_count_before_log, iban_count_before_log, tx_count_this_month_so_far

This section specifies the generation procedure, preprocessing, and evaluation split of the Synthetic Business Transaction dataset.

Overview. The dataset models business-to-business (B2B) payment activity for nine customers over a fixed time period. It comprises 6,364 transactions with 50 labeled anomalies, corresponding to an anomaly ratio $\rho = 50/6364 \approx 0.786\%$, and is constructed to assess detection performance under extreme class imbalance.

Nominal behavior. Each customer is associated with one primary recurring payment series to a fixed business partner, together with five to ten additional background series to further partners. Each series consists of monthly payments whose amounts follow a Gaussian distribution around a series-specific base value, with per-series payment terms and mild multiplicative growth or decline over time. In addition, each customer issues sporadic one-off payments. These regularities define the nominal distribution against which anomalies are measured.

Anomaly injection. Anomalies are drawn from seven types: amount spikes ($1.5\text{--}2\times$ the expected value), duplicate payments within a series, modified payee information, temporal shifts, atypical payment channels (card/mobile in place of the usual method), IBAN inconsistencies relative to series history, and subtle payee-name perturbations accompanied by an IBAN change (e.g., “Solutions” $\rightarrow$ “Systems”). For each customer, a single anomaly type is selected at random and applied a random number of times to that customer’s primary series; the remaining series are left nominal. This per-customer procedure yields a total of 50 anomalies across the dataset.

Preprocessing. Each transaction is encoded by 21 numerical features ². All features are computed in temporal order, such that the representation of a transaction depends solely on preceding transactions; mismatch flags, in particular, compare the current value against the modal historical value of the same series. This sequential construction precludes information leakage from future observations.

Training and Evaluation split. The data is partitioned chronologically at its temporal midpoint (by day), yielding 3,188 training transactions (28 anomalies) and 3,176 test transactions (22 anomalies).

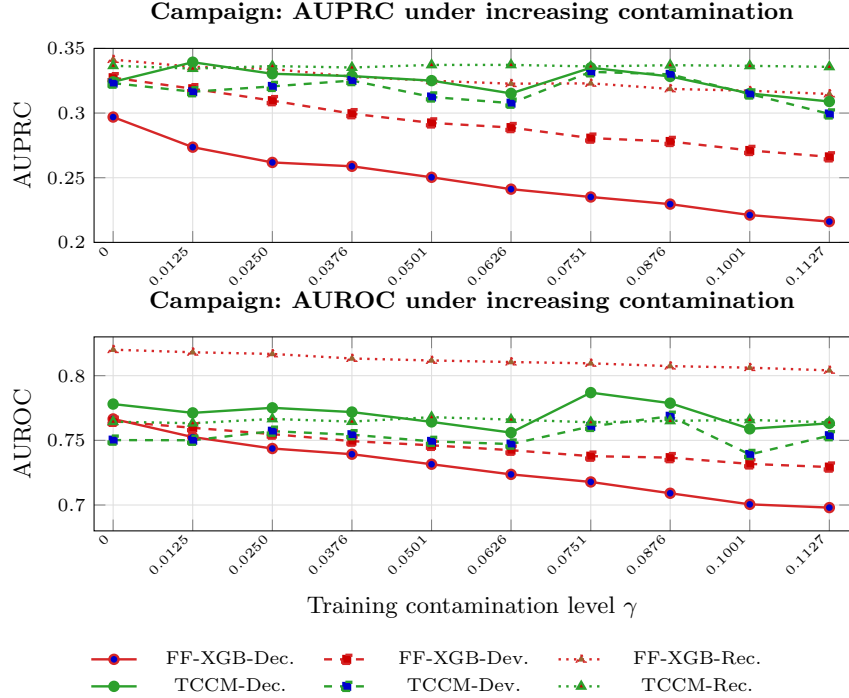


Fig. 2: Campaign contamination curves for AUPRC and AUROC. Campaign contains 41,188 samples with 4,640 anomalies, giving an anomaly ratio $\rho = 0.1127$ or 11.27%. The x-axis reports the training contamination level γ as a fraction, where $\gamma = 0$ means anomaly-free training data, $\gamma = 0.0125$ means approximately 1.25% contamination, and $\gamma = 0.1127 \approx \rho$ corresponds to full contamination. We compare TCCM with Forest-Flow using XGBoost, and plot Decision, Deviation, and Reconstruction anomaly scores using the mean values from the Campaign table.

D Additional Contamination Curves

Figures 2, 3, and 4 show AUROC and AUPRC trends across all training contamination levels for Campaign, Synthetic Business Transaction, and Waveform, respectively. The x-axis denotes the contamination level γ , where $\gamma = 0$ indicates clean training data and the final point corresponds to the full-contamination setting. For each dataset, we compare TCCM, Forest-Flow with XGBoost, using Decision, Deviation, and Reconstruction scores, complementing Table 1 in main paper with performance trends beyond the zero- and full-contamination endpoints.

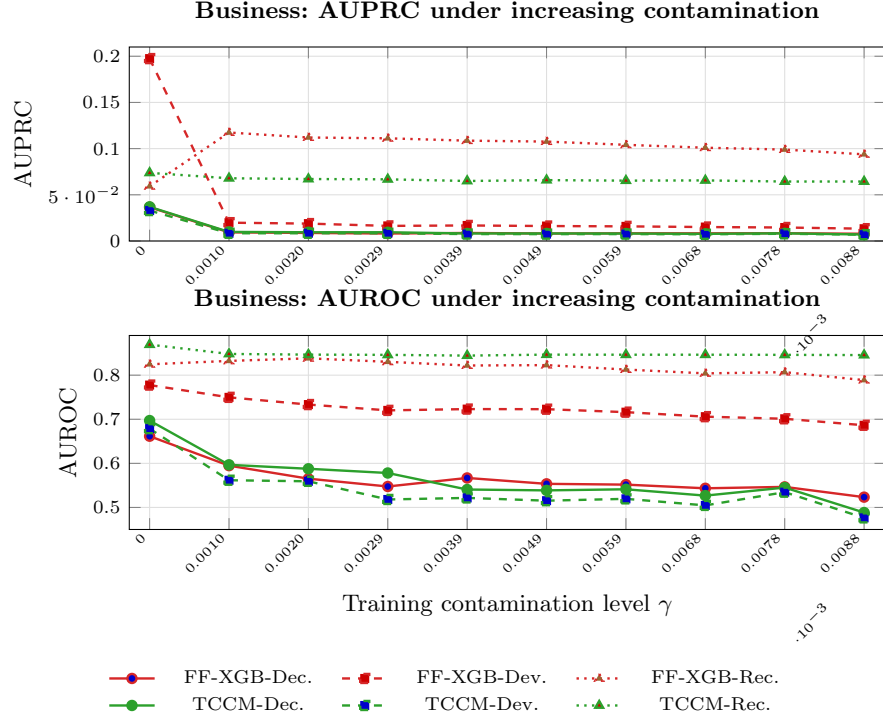


Fig. 3: Business dataset contamination curves for AUPRC and AUROC. The Business dataset contains 6,364 samples with 50 anomalies, corresponding to a dataset-level anomaly ratio of $\rho = 50/6364 \approx 0.00786$ or 0.786%. The x-axis reports the training contamination level γ as a fraction, where $\gamma = 0$ means anomaly-free training data and, for example, $\gamma = 0.0009759$ means approximately 0.0976% contamination. The final plotted level, $\gamma = 0.0087829$, corresponds to the full-contamination setting used in this training split. We compare TCCM with Forest-Flow using XGBoost, and plot Decision, Deviation, and Reconstruction anomaly scores using the mean values from the Business table.

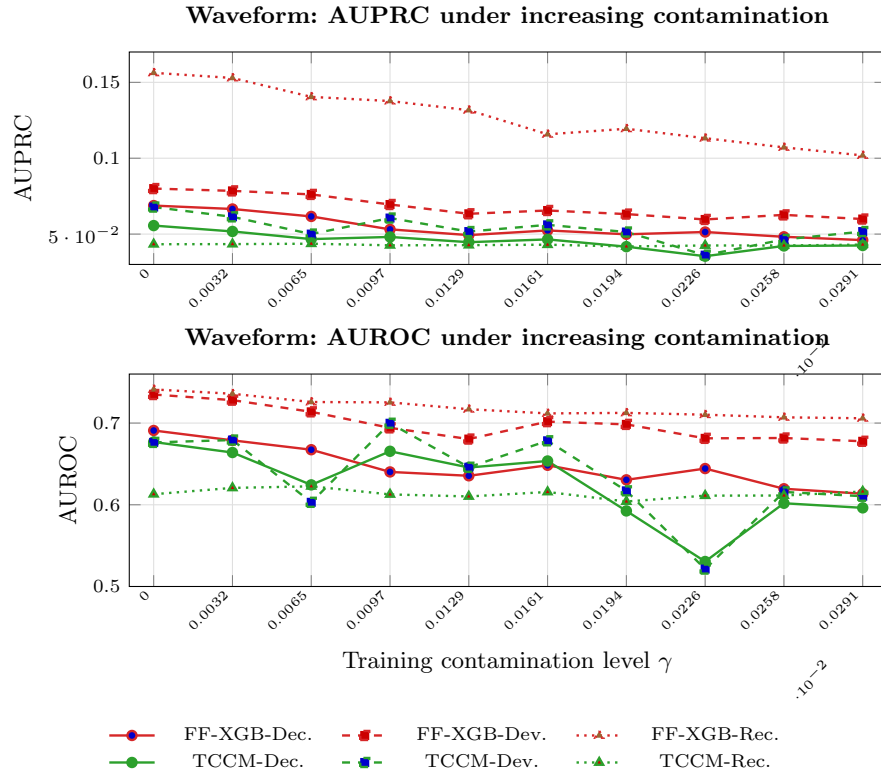


Fig. 4: Waveform contamination curves for AUPRC and AUROC. Waveform has an anomaly ratio of approximately $\rho = 0.0291$ or 2.90%. The x-axis reports the training contamination level γ as a fraction, where $\gamma = 0$ means anomaly-free training data, $\gamma = 0.0032$ means approximately 0.32% contamination, and $\gamma = 0.0291 \approx \rho$ corresponds to full contamination. We compare TCCM with Forest-Flow using XGBoost, and plot Decision, Deviation, and Reconstruction anomaly scores.

Bibliography

- [1] Breunig, M.M., Kröger, P., Ng, R.T., Sander, J.: LOF: Identifying density-based local outliers. In: Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data. pp. 93–104 (2000)
- [2] Chen, J., Sathe, S., Aggarwal, C.C., Turaga, D.: Outlier detection with autoencoder ensembles. In: Proceedings of the 2017 SIAM International Conference on Data Mining (SDM). pp. 90–98 (2017)
- [3] Chen, T., Guestrin, C.: Xgboost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 785–794. ACM (2016). <https://doi.org/10.1145/2939672.2939785>
- [4] Ding, X., Zhao, L., Akoglu, L.: Hyperparameter sensitivity in deep outlier detection: Analysis and a scalable hyper-ensemble solution. *Advances in Neural Information Processing Systems* **35**, 9603–9616 (2022)
- [5] Fernando, T., Gammulle, H., Denman, S., Sridharan, S., Fookes, C.: Deep learning for medical anomaly detection—a survey. *ACM Computing Surveys (CSUR)* **54**(7), 1–37 (2021)
- [6] Han, S., Hu, X., Huang, H., Jiang, M., Zhao, Y.: Adbench: Anomaly detection benchmark. *Advances in neural information processing systems* **35**, 32142–32159 (2022)
- [7] Hilal, W., Gadsden, S., Yawney, J.: Financial fraud: A review of anomaly detection techniques and recent advances. *Expert Systems with Applications* **193**, 116429 (12 2022). <https://doi.org/10.1016/j.eswa.2021.116429>
- [8] Huang, Y., Zhang, Y., Wang, L., Zhang, F., Lin, X.: Entropystop: Unsupervised deep outlier detection with loss entropy. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 1143–1154 (2024)
- [9] Jolicoeur-Martineau, A., Fatras, K., Kachman, T.: Forestdiffusion: Generating and imputing tabular data via diffusion and flow-based gradient-boosted trees - GitHub repository. <https://github.com/SamsungSAILMontreal/ForestDiffusion> (2024), zugriff: 13. Dezember 2025
- [10] Jolicoeur-Martineau, A., Fatras, K., Kachman, T.: Generating and imputing tabular data via diffusion and flow-based gradient-boosted trees. In: International conference on artificial intelligence and statistics. pp. 1288–1296. PMLR (2024)
- [11] Kobyzev, I., Prince, S.J., Brubaker, M.A.: Normalizing flows: An introduction and review of current methods. *IEEE transactions on pattern analysis and machine intelligence* **43**(11), 3964–3979 (2020)
- [12] Lee, S., Kim, B., Ye, J.C.: Minimizing trajectory curvature of ode-based generative models. In: International Conference on Machine Learning. pp. 18957–18973. PMLR (2023)
- [13] Li, Z., Huang, Q., Zhu, Y., Yang, L., Amiri, M.M., van Stein, N., van Leeuwen, M.: Scalable, explainable and provably robust anomaly detection

- with one-step flow matching. In: The Thirty-ninth Annual Conference on Neural Information Processing Systems (2025), <https://openreview.net/forum?id=jDYuadVajk>
- [14] Li, Z., Huang, Q., Zhu, Y., Yang, L., Amiri, M.M., van Stein, N., van Leeuwen, M.: Tccm: Time-conditioned contraction matching - GitHub repository. <https://github.com/ZhongLIFR/TCCM-NIPS> (2025), accessed: 18.12.2025
 - [15] Lipman, Y., Chen, R.T.Q., Ben-Hamu, H., Nickel, M., Le, M.: Flow matching for generative modeling. In: The Eleventh International Conference on Learning Representations (2023), <https://openreview.net/forum?id=PqvMRDCJT9t>
 - [16] Liu, X., Gong, C., Liu, Q.: Flow straight and fast: Learning to generate and transfer data with rectified flow. arXiv preprint arXiv:2209.03003 (2022)
 - [17] Liu, Y., Li, Z., Zhou, C., Jiang, Y., Sun, J., Wang, M., He, X.: Generative adversarial active learning for unsupervised outlier detection. *IEEE Transactions on Knowledge and Data Engineering* **32**(8), 1517–1528 (2019)
 - [18] Livernoche, V., Jain, V., Hezaveh, Y., Ravanbakhsh, S.: On diffusion modeling for anomaly detection. In: The Twelfth International Conference on Learning Representations (2024), <https://openreview.net/forum?id=1R3rk7ysXz>
 - [19] Maziarka, Ł., Śmieja, M., Sendera, M., Struski, Ł., Tabor, J., Spurek, P.: Oneflow: One-class flow for anomaly detection based on a minimal volume region. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **44**(11), 8508–8519 (2021)
 - [20] McDermott, M., Zhang, H., Hansen, L., Angelotti, G., Gallifant, J.: A closer look at auroc and auprc under class imbalance. *Advances in Neural Information Processing Systems* **37**, 44102–44163 (2024)
 - [21] Pang, G., Shen, C., Cao, L., Hengel, A.V.D.: Deep learning for anomaly detection: A review. *ACM Comput. Surv.* **54**(2) (Mar 2021). <https://doi.org/10.1145/3439950>, <https://doi.org/10.1145/3439950>
 - [22] Qiu, C., Pfrommer, T., Kloft, M., Mandt, S., Rudolph, M.: Neural transformation learning for deep anomaly detection beyond images. In: International conference on machine learning. pp. 8703–8714. PMLR (2021)
 - [23] Ruff, L., Vandermeulen, R., Goernitz, N., Deecke, L., Siddiqui, S.A., Binder, A., Müller, E., Kloft, M.: Deep one-class classification. In: Proceedings of the 35th International Conference on Machine Learning. pp. 4393–4402 (2018)
 - [24] Schlegl, T., Seeböck, P., Waldstein, S.M., Schmidt-Erfurth, U., Langs, G.: Unsupervised anomaly detection with generative adversarial networks to guide marker discovery. In: Niethammer, M., Styner, M., Aylward, S., Zhu, H., Oguz, I., Yap, P.T., Shen, D. (eds.) *Information Processing in Medical Imaging*. pp. 146–157. Springer International Publishing, Cham (2017)
 - [25] Schölkopf, B., Williamson, R.C., Smola, A., Shawe-Taylor, J., Platt, J.: Support vector method for novelty detection. In: Solla, S., Leen, T., Müller, K. (eds.) *Advances in Neural Information Processing Systems*. vol. 12. MIT Press (1999), https://proceedings.neurips.cc/paper_files/paper/1999/file/8725fb777f25776ffa9076e44fcfd776-Paper.pdf

- [26] Shenkar, T., Wolf, L.: Anomaly detection for tabular data with internal contrastive learning. In: International Conference on Learning Representations (2022), https://openreview.net/forum?id=_hszZbt46bT
- [27] Shyu, M.L., Chen, S.C., Sarinnapakorn, K., Chang, L.: A novel anomaly detection scheme based on principal component classifier. In: Proceedings of International Conference on Data Mining (01 2003)
- [28] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. *Advances in neural information processing systems* **30** (2017)
- [29] Wang, H., Pang, G., Shen, C., Ma, C.: Unsupervised representation learning by predicting random distances. *arXiv preprint arXiv:1912.12186* (2019)
- [30] Yu, J., Zhang, Y.: Challenges and opportunities of deep learning-based process fault detection and diagnosis: a review. *Neural Computing and Applications* **35**(1), 211–252 (2023)
- [31] Zhou, C., Paffenroth, R.C.: Anomaly detection with robust deep autoencoders. In: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 665–674. ACM (2017). <https://doi.org/10.1145/3097983.3098052>
- [32] Zong, B., Song, Q., Min, M.R., Cheng, W., Lumezanu, C., Cho, D., Chen, H.: Deep autoencoding gaussian mixture model for unsupervised anomaly detection. In: International Conference on Learning Representations (2018), <https://openreview.net/forum?id=BJJLHbb0->